LAPORAN PROGRES TUGAS BESAR CLOUD COMPUTING

Kompilasi Laporan Progres: Pertemuan 1 & Pertemuan 2

Mata Kuliah: Cloud Computing (3 SKS) • Semester 7

Topik Proyek: **Auto-Scaling & Load Balancing Sistem War Tiket Menggunakan Docker Swarm**

Studi Kasus: Sistem War Tiket Konser Musik (Flash Crowd)

Repositori Git: github.com/Cahyooo69/Auto-Scaling-Load-Balancing

Kelompok 1: H7A.7 (Kamis Pagi) • Dosen Pengampu: **Dr. Moch Yasin, S.Kom., M.Kom., M.B.A., CDCP**

- **Muhammad Ilham Nurcahyo** (09040623063)
- **Muhammad Daud Rizaldiansyah** (09040623061)
- **Hafid Aji Fauzan** (0920623027)
- **Davin Aryaguna Hermansyah** (09020623020)

BAGIAN I — RINGKASAN PROGRES PERTEMUAN 1: PLANNING & ARCHITECTURE

1. LATAR BELAKANG, PROBLEM STATEMENT & TARGET SLA

Trafik *War Tiket* memiliki karakteristik **flash crowd**, di mana ribuan pengguna mengirim permintaan transaksi dalam detik yang sama. Arsitektur konvensional berisiko mengalami *crash OOM*, kegagalan transaksi, dan *race condition* (overselling). Proyek ini merancang solusi terorkestrasi berbasis Docker Swarm dengan sasaran SLA ketat: **Latency < 200ms**, **Error Rate < 1%**, serta **0% Overselling** di bawah batasan komputasi lokal `0.25 CPU` dan `256M RAM` per container.

2. DESAIN ARSITEKTUR SISTEM 3-TIER

Trafik Klien / Beban Uji → HTTP Port 80 (Gerbang Tunggal Ingress)

1. Ingress Layer (Nginx)
- Reverse Proxy & Load Balancer
- Port Host: `80:80`
- Swarm VIP: `127.0.0.11`
- Limit: `0.25 CPU` | `256M RAM`

2. Application Layer (App)
- Node.js Mock API Worker
- Dynamic Scaling:
2 → 8 Replika
- Delay Simulasi: 50–100ms
- Limit: `0.25 CPU` | `256M RAM`

3. State Layer (Redis)
- In-Memory Datastore
-
Atomic Lua Scripting
- Zero Race Condition & Oversell
- Limit: `0.25 CPU` | `256M RAM`

Komponen Service	Image & Port	Alokasi Sumber Daya	Fungsi Utama Arsitektur
wartiket_nginx	`wartiket-nginx` (80:80)	**CPU: 0.25 | RAM: 256M**	Reverse proxy gerbang masuk, round-robin load balancer, dynamic DNS failover.
wartiket_app	`wartiket-app` (Port 3000 Internal)	**CPU: 0.25 | RAM: 256M**	Worker pemroses checkout tiket, skala horizontal 2 s.d 8 replika.
wartiket_redis	`redis:7-alpine` (Port 6379 Internal)	**CPU: 0.25 | RAM: 256M**	Penyimpan kuota tiket dengan eksekusi atomik Lua script dan persistensi AOF.

3. RINGKASAN KETERCAPAIAN LUARAN PERTEMUAN 1

Target Luaran Wajib Pertemuan 1	Hasil Verifikasi Dokumen & Kode	Status Capaian
Diagram Arsitektur Sistem 3-Tier	Selesai dan terdokumentasi lengkap pada diagram Mermaid di `README.md`.	**Selesai 100%**
Skema Jaringan & Isolasi Port	Docker Overlay Network `wartiket-net` dengan proteksi port internal backend.	**Selesai 100%**
Spesifikasi `docker-compose.yml`	Lengkap dengan deklarasi deploy Swarm, limits `0.25 CPU` dan `256M RAM`.	**Selesai 100%**
Repositori Git Kelompok	Terkonfigurasi dan aktif di GitHub: `Cahyooo69/Auto-Scaling-Load-Balancing`.	**Selesai 100%**

BAGIAN II — PROGRES PERTEMUAN 2: ENVIRONMENT SETUP & CORE DEPLOYMENT

Luaran Wajib Pertemuan 2 (Target Silabus):
"Seluruh container dapat berjalan (up & running) dalam jaringan internal Docker dengan resource limit yang sudah diterapkan."

4. BUKTI OPERASIONAL SELURUH CONTAINER (UP & RUNNING)

Seluruh service pada berkas `docker-compose.yml` telah berhasil di-deploy ke klaster Docker Swarm lokal bernama `wartiket`. Status operasional 100% aktif (Healthy/Running) tanpa ada restart berulang:

Service Name	Container ID	Status Desired	Current State	Port Binding	Fungsi Arsitektur
`wartiket_nginx`	`i1orif57jdon`	Running	Running (Healthy)	`*:80->80/tcp`	Reverse Proxy & Ingress Load Balancer (Port publik gerbang masuk).
`wartiket_app.1`	`ri2ub2cbty8t`	Running	Running (Healthy)	`3000/tcp` (Internal)	Replika Worker Backend 1 (Mock API checkout tiket delay 50-100ms).
`wartiket_app.2`	`w06pvpk878z9`	Running	Running (Healthy)	`3000/tcp` (Internal)	Replika Worker Backend 2 (Menyeimbangkan beban bersama replika 1).
`wartiket_redis`	`wvtwrcqh4wks`	Running	Running (Healthy)	`6379/tcp` (Internal)	In-Memory Datastore & Atomic Lua Script Locking (Stok persistent).

```
$ docker stack services wartiket
ID             NAME             MODE         REPLICAS   IMAGE                  PORTS
lbuec3bv49sx   wartiket_app     replicated   2/2        wartiket-app:latest
x7zlrm1hedl3   wartiket_nginx   replicated   1/1        wartiket-nginx:latest  *:80->80/tcp
50k0j53unes3   wartiket_redis   replicated   1/1        redis:7-alpine
```

5. VERIFIKASI PENERAPAN RESOURCE LIMITS DI LEVEL KERNEL

Ketentuan standar Cloud Computing (3 SKS) mewajibkan pembatasan sumber daya ketat pada setiap container:

- **Maksimal CPU:** `0.25 core` (maksimal 25% dari 1 core CPU host).
- **Maksimal Memori RAM:** `256 MB` (268.435.456 Bytes).
- **Reservasi Sumber Daya:** `0.10 core` CPU dan `64 MB` RAM.

Penerapan batas ini diverifikasi langsung dari Docker Daemon menggunakan inspeksi metadata tugas Swarm:

Service	Spesifikasi Compose	Nilai Inspeksi Kernel (Docker Inspect)	Status Kepatuhan
Backend API (`wartiket_app`)	CPU: 0.25 | RAM: 256M	`NanoCPUs: 250000000 | MemoryBytes: 268435456`	Memenuhi Standar
Load Balancer (`wartiket_nginx`)	CPU: 0.25 | RAM: 256M	`NanoCPUs: 250000000 | MemoryBytes: 268435456`	Memenuhi Standar
Database Redis (`wartiket_redis`)	CPU: 0.25 | RAM: 256M	`NanoCPUs: 250000000 | MemoryBytes: 268435456`	Memenuhi Standar

Bukti Eksekusi Terminal (Docker Inspect):

```
$ docker service inspect wartiket_app --format "{{json .Spec.TaskTemplate.Resources}}"
{"Limits":{"NanoCPUs":250000000,"MemoryBytes":268435456},"Reservations":{"NanoCPUs":100000000,"MemoryBytes":67108864}}
```

Keterangan: 250.000.000 NanoCPUs setara tepat dengan 0.25 CPU Core, dan 268.435.456 Bytes setara tepat dengan 256 MB RAM.

6. SKEMA JARINGAN INTERNAL & ISOLASI DOCKER OVERLAY

Seluruh container berkomunikasi di dalam **Docker Overlay Network** bernama `wartiket_wartiket-net`:

- **Alokasi Subnet:** `10.0.1.0/24` terisolasi dengan gateway internal otomatis.
- **Isolasi Port:** Port 3000 (Node.js) dan Port 6379 (Redis) tidak dipublikasikan ke host, sehingga tertutup rapat dari akses internet publik dan hanya dapat dijangkau oleh Nginx.
- **Service Discovery:** Nginx me-resolve nama host `app:3000` melalui DNS internal Swarm (`127.0.0.11`).

```
$ docker network inspect wartiket_wartiket-net --format "Driver: {{.Driver}} | Scope: {{.Scope}} | Subnet: {{range .IPAM.
Config}}{{.Subnet}}{{end}}"
Driver: overlay | Scope: swarm | Subnet: 10.0.1.0/24
```

7. PENGUJIAN FUNGSIONAL END-TO-END JARINGAN

Validasi konektivitas antar-container dibuktikan melalui pengujian 20 transaksi pembelian tiket bersamaan via port 80 Nginx menggunakan skrip penguji `node test-quick.js`:

```
$ node test-quick.js
Sending 20 concurrent checkout requests to http://localhost/tickets/buy...
┌─────────┬────┬────────┬─────────┬──────────────────┬───────────┬─────────┬──────────────────────────────┐
│ (index) │ ID │ Status │ Latency │ ServedBy         │ Remaining │ Success │ Info                         │
├─────────┼────┼────────┼─────────┼──────────────────┼───────────┼─────────┼──────────────────────────────┤
│ 0       │ 1  │ 200    │ '115ms' │ '86d0051a2950' │ 999       │ true    │ 'Tiket berhasil didapatkan!' │
│ 1       │ 2  │ 200    │ '90ms'  │ '83f7bdab3528' │ 998       │ true    │ 'Tiket berhasil didapatkan!' │
│ ...     │ .. │ 200    │ ...     │ ...              │ ...       │ true    │ 'Tiket berhasil didapatkan!' │
│ 19      │ 20 │ 200    │ '98ms'  │ '86d0051a2950' │ 980       │ true    │ 'Tiket berhasil didapatkan!' │
└─────────┴────┴────────┴─────────┴──────────────────┴───────────┴─────────┴──────────────────────────────┘
--- Load Balancing Distribution Across Containers ---
┌──────────────┬────────┐
│ (index)      │ Values │
├──────────────┼────────┤
│ 86d0051a2950 │ 10     │
│ 83f7bdab3528 │ 10     │
└──────────────┴────────┘
```

Kesimpulan Hasil Pengujian Fungsional:

1. **Konektivitas Nginx → Backend:** Berhasil 100%. Request terdistribusi seimbang **50:50** (10 request ke Worker 1 dan 10 request ke Worker 2).
2. **Konektivitas Backend → Redis:** Berhasil 100%. Script Lua atomic decrement mengeksekusi stok secara akurat dari 999 ke 980 tanpa race condition.
3. **Target SLA:** Latency rata-rata ~95ms, jauh lebih cepat daripada batas toleransi SLA yaitu 200ms.

8. MATRIKS PEMENUHAN TARGET LUARAN PERTEMUAN 2

Indikator Keberhasilan Pertemuan 2	Target Persyaratan Silabus	Hasil Verifikasi Lapangan	Status Capaian
Status Container	Seluruh container up & running	4 container aktif (Nginx, App.1, App.2, Redis) status *Running*.	**Selesai 100%**
Isolasi Jaringan	Jaringan internal Docker terisolasi	Docker Overlay Network (`10.0.1.0/24`) dengan port internal terproteksi.	**Selesai 100%**
Penerapan Resource Limit	Maksimal 0.25 CPU & 256MB RAM	Terverifikasi pada Docker Daemon (`250000000 NanoCPUs` & `256M`).	**Selesai 100%**
End-to-End Core Deployment	Layanan terintegrasi & responsif	Uji 20 transaksi sukses 100% dengan round-robin load balancing.	**Selesai 100%**

BAGIAN III — ANALISIS KEANDALAN SISTEM & MITIGASI SPOF

> **Tindak Lanjut Masukan Dosen (Dr. Moch Yasin, S.Kom., M.Kom., M.B.A., CDCP):**
> "*Terima kasih, tinggal SPOF diselesaikan*" — Catatan evaluasi High Availability (HA) arsitektur sistem.

9. IDENTIFIKASI SINGLE POINT OF FAILURE (SPOF) SAAT INI

SPOF (*Single Point of Failure*) adalah titik tunggal dalam arsitektur yang apabila mengalami kegagalan proses/koneksi, berpotensi melumpuhkan seluruh fungsionalitas sistem. Analisis titik SPOF pada arsitektur klaster kami:

Komponen Service	Status Replikasi	Klasifikasi SPOF	Dampak Jika Terjadi Kegagalan Container
Backend API (app)	`replicas: 2` (Scalable s.d 8)	**Bukan SPOF**	Toleran terhadap kegagalan. Jika 1 worker mati, request langsung dialihkan ke worker lain tanpa downtime.
Reverse Proxy (`nginx`)	`replicas: 1`	**Titik SPOF 1**	Jika Nginx mati, seluruh akses HTTP dari klien eksternal terputus total.
Datastore (`redis`)	`replicas: 1`	**Titik SPOF 2**	Jika Redis mati, endpoint checkout tiket menolak transaksi (HTTP 500) demi mencegah data minus.

10. MITIGASI LEVEL 1 YANG SUDAH DITERAPKAN (TAHAP SAAT INI)

Pada tahapan Pertemuan 2, sistem telah dilengkapi mekanisme mitigasi dasar untuk meredam dampak kegagalan:

- **Self-Healing Otomatis Docker Swarm:** Konfigurasi `restart_policy: condition: on-failure` memastikan bahwa jika container Redis atau Nginx mengalami crash mendadak, Docker Swarm akan langsung men-spawn ulang container baru dalam hitungan detik.
- **Data Persistence (AOF & Volume):** Redis dikonfigurasi dengan flag `--appendonly yes` yang dipetakan ke volume Docker `redis-data:/data`. Saat Redis hidup kembali, seluruh riwayat transaksi dan sisa stok tiket tetap utuh (zero data loss).
- **Fail-Safe API Circuit Breaker:** Driver `ioredis` dikonfigurasi dengan `enableOfflineQueue: false` dan blok *try-catch* aman. Saat Redis mati, transaksi langsung ditolak dengan HTTP 500 (Fail-Safe), sehingga tidak terjadi transaksi gantung atau *overselling*. Driver otomatis melakukan rekoneksi saat Redis kembali aktif.

11. ROADMAP SOLUSI HIGH AVAILABILITY SEJATI (LEVEL 2: RENCANA LANJUTAN)

Untuk mengeliminasi titik SPOF secara mutlak menuju arsitektur *High Availability* (HA) sejati pada pertemuan lanjutan:

Titik SPOF	Solusi Arsitektur HA yang Direncanakan	Mekanisme Kerja Failover
Nginx Ingress	Multi-Replica Nginx Ingress & Swarm Routing Mesh	Menaikkan replika Nginx menjadi 2 instance. Port 80 dipublikasikan melalui Swarm Ingress Routing Mesh, sehingga kegagalan satu Nginx tidak memutus akses klien.
Redis Datastore	Redis Sentinel Architecture (Master-Replica)	Mengonfigurasi 1 Redis Master untuk penulisan stok, 1 Redis Replica untuk sinkronisasi data, dan Redis Sentinel untuk promosi failover otomatis tanpa downtime.

12. KESIMPULAN PROGRES

Seluruh target penugasan **Pertemuan 1 (Planning & Architecture)** dan **Pertemuan 2 (Environment Setup & Resource Limits)** telah **100% tuntas dan terverifikasi di lapangan**. Sistem siap didemonstrasikan di hadapan dosen pengampu, dan mitigasi SPOF siap dilanjutkan pada fase High Availability berikutnya.